TRƯỜNG ĐẠI HỌC SÀI GÒN KHOA CÔNG NGHỆ THÔNG TIN

KIỂM THỬ PHẦN MỀM

Assignment 1 - Kiểm Thử Hộp Đen

Thiết kế test case và tự động hóa web với JUnit 5

GVHD: Từ Lãng Phiêu
SV - MSSV
Huỳnh Trọng Nhân - 3123560055

TP. HỒ CHÍ MINH, THÁNG 10/2026

0.1 Mục lục

1. Phần giới thiệu
    1. Xác định số lượng sinh viên trong tập mẫu
    2. Các công trình liên quan
2. NV2 - Phân tích yêu cầu
    1. Đặc tả yêu cầu
    2. Đánh giá rủi ro
    3. Ma trận truy vết
3. NV3 - Thiết kế test case
    1. Phân lớp tương đương
    2. Phân tích giá trị biên
    3. NFR-01 hiệu năng
    4. Tổng số test case
4. NV4 - thực thi thủ công và báo lỗi

5. NV5 – Tự động hóa bằng JUnit 5

1 Phần giới thiệu

1. **Xác định số lượng sinh viên trong tập mẫu**
2. **Các công trình liên quan**

   Giới thiệu vấn đề Công việc các bạn cần làm

2 NV2 – Phân tích yêu cầu

2.1 Đặc tả yêu cầu

2.1.1 FR-01 – Đăng nhập

Mô tả yêu cầu:

Người dùng nhập username và password để đăng nhập vào hệ thống. Username phân biệt chữ hoa/chữ thường. Nếu đăng nhập thành công, hệ thống chuyển đến trang `inventory.html`. Nếu đăng nhập thất bại, hệ thống hiển thị thông báo lỗi tương ứng.

Các quy tắc kiểm tra:

- Nếu username để trống, hệ thống hiển thị `Username is required`.
- Nếu username đã được nhập nhưng password để trống, hệ thống hiển thị `Password is required`.
- Nếu username hoặc password không khớp với thông tin của tài khoản hợp lệ, hệ thống hiển thị `Username and password do not match any user in this service`.
- Nếu thông tin đăng nhập đúng nhưng tài khoản đã bị khóa, hệ thống hiển thị `Sorry, this user has been locked out.`.
- Nếu thông tin đăng nhập hợp lệ và tài khoản đang hoạt động, hệ thống chuyển đến trang `inventory.html`.

2.1.2 FR-02 – Giỏ hàng

Mô tả yêu cầu:

Mỗi sản phẩm có nút `Add to cart`/`Remove`. Badge trên biểu tượng giỏ hàng hiển thị số sản phẩm (1–6); khi giỏ rỗng, badge không hiển thị.

Các quy tắc kiểm tra:

- Khi thêm một sản phẩm, nút tương ứng chuyển thành `Remove`.
- Badge hiển thị số sản phẩm hiện có trong giỏ.

- Khi xóa một sản phẩm, badge được cập nhật tương ứng.
- Khi giỏ rỗng, badge không hiển thị.

2.1.3 FR-03 – Thanh toán

Mô tả yêu cầu:

First Name, Last Name, Postal Code đều bắt buộc, kiểm tra theo thứ tự đó. Trang tổng quan hiển thị Item total, Tax = 8% Item total (làm tròn 2 chữ số) và Total = Item total + Tax. Nhấn Finish hiển thị Thank you for your order! và làm rỗng giỏ hàng. Nhấn Cancel ở bước nhập thông tin quay về cart.html.

Các quy tắc kiểm tra

- Nếu First Name trống,hiển thị `First Name is required`.
- Nếu Last Name trống,hiển thị `Last Name is required`.
- Nếu Postal Code trống, hiển thị `Postal Code is required`.
- Nếu nhiều trường trống, kiểm tra theo thứ tự và hiển thị lỗi trường thiếu đầu tiên.
- Nếu cả ba trường đều có dữ liệu, nhấn `Continue`, chuyển qua trang tổng quan.
- Hiển thị Item total, bảng tổng giá các sản phẩm trong giỏ hàng.
- Hiển thị Tax và Total, `Tax = ItemTotal * 8%`, làm tròn đến 2 chữ số thập phân;`Total = ItemTotal + Tax`.
- Nhấn Finish, hiển thị `Thank you for your order!`, làm rỗng giỏ hàng.
- Nhấn Cancel ở bước nhập thông tin, quay về `cart.html`,các sản phẩn vẫn còn trong giỏ.

2.1.4 FR-04 – Sắp xếp

Mô tả yêu cầu:

Tại trang sản phẩm (`inventory.html`), người dùng sắp xếp danh sách sản phẩm bằng danh sách thả xuống (dropdown) ở góc phải phía trên danh sách. Dropdown có 4 lựa chọn: `Name (A to Z)`, `Name (Z to A)`, `Price (low to high)` và `Price (high to low)`.

Các quy tắc kiểm tra:

- Khi mới vào trang (chưa chọn gì), danh sách ở kiểu `Name (A to Z)`.
- Chọn `Name (A to Z)`: tên sản phẩm được xếp tăng dần theo bảng chữ cái.
- Chọn `Name (Z to A)`: tên sản phẩm được xếp giảm dần theo bảng chữ cái.

- Chọn `Price (low to high)`: giá sản phẩm được xếp tăng dần theo giá trị số.
- Chọn `Price (high to low)`: giá sản phẩm được xếp giảm dần theo giá trị số.
- Sau mỗi lần sắp xếp, danh sách vẫn hiển thị đủ 6 sản phẩm; chỉ thay đổi thứ tự, không thay đổi tên hay giá của từng sản phẩm.

2.1.5 NFR-01 – Hiệu năng

Mô tả yêu cầu: Đảm bảo tốc độ phản hồi và thời gian tải trang sản phẩm (`inventory.html`) đạt tiêu chuẩn khi người dùng thực hiện thao tác đăng nhập thành công.

Các quy tắc kiểm tra:

- **Thời gian phản hồi:** Sau khi login đúng, trang sản phẩm (`inventory.html`) phải hiển thị hoàn tất trong vòng ≤ 3 giây.
- **Phương pháp đo đạc:**
  - Giá trị đánh giá được tính là trung vị (median) của tổng cộng 5 lần đo.
  - Mỗi lần đo phải thực hiện một phiên đăng nhập mới hoàn toàn (bắt buộc xóa cookie/phiên làm việc trước đó).
  - Điều kiện môi trường: Kiểm thử trong điều kiện mạng ổn định.

2.2 Đánh giá rủi ro

Table 1: Đánh giá rủi ro

Mã	P	I	R	Số TC	Lý do
FR-01	3	4	12	–	Username hoặc password để trống, thông tin không hợp lệ, username phân biệt chữ hoa/thường và tài khoản bị khóa, hệ thống có thể chấp nhận thông tin đăng nhập không hợp lệ.
FR-02	3	3	9	7	Badge hiển thị sai số lượng hoặc nút Add to cart/Remove không cập nhật có thể gây nhầm lẫn về sản phẩm trong giỏ hàng.
FR-03	3	5	15	–	Có nhiều nhánh kiểm tra dữ liệu và thao tác Continue/Cancel/Finish; dễ sai thứ tự báo lỗi, tính thuế hoặc tổng tiền. Sai sót ảnh hưởng trực tiếp đến số tiền và tính chính xác của đơn hàng.
FR-04	2	3	6	–	Chức năng chỉ có 4 lựa chọn trong một dropdown nên ít nhánh xử lý. Sắp xếp sai gây khó chịu, và nếu giá hiển thị sai (ví dụ ở tài khoản có lỗi giao diện) thì người dùng có thể hiểu nhầm khi chọn mua.
NFR-01	4	3	12	1	Tài khoản `performance_glitch_user` bị làm chậm có chủ đích ($P = 4$), người dùng tưởng hệ thống treo và thoát ra ($I = 3$).

2.3 Ma trận truy vết

Table 2: Ma trận truy vết yêu cầu

Yêu cầu	Kỹ thuật	Test case	Lớp test	Lỗi
FR-01	EP, BVA, DT	TC-LOGIN-EP1–EP8, TC-LOGIN-B1–B6, TC-LOGIN-DT1–DT5	LoginTest	–
FR-02	EP, BVA	TC-CART-B1–B5, TC-CART-EP1–2	CartTest	–
FR-03	UC, DT	TC-CHK-01–05	CheckoutTest	–
FR-04	EP	TC-SORT-01–06	SortTest	BUG
NFR-01	Đo thời gian	TC-NFR-01	PerformanceTest	BUG

3 NV3 - Thiết kế test case

3.1 Phân lớp tương đương (EP)

3.1.1 EP – Phân lớp tương đương cho FR-01

Đối với FR-01, các trường hợp đăng nhập được chia thành các lớp tương đương dựa trên tính hợp lệ của username, password và trạng thái tài khoản.

- **EC1 – Tài khoản hợp lệ:** Username tồn tại và tài khoản đang hoạt động.
- **EC2 – Tài khoản bị khóa:** Username tồn tại nhưng tài khoản đã bị khóa.
- **EC3 – Username rỗng:** Người dùng không nhập username.
- **EC4 – Username không tồn tại:** Username không thuộc tài khoản hợp lệ trong hệ thống.
- **EC5 – Username sai chữ hoa/thường:** Username không khớp do phân biệt chữ hoa và chữ thường.
- **EC6 – Password hợp lệ:** Password trùng khớp với thông tin của tài khoản đang hoạt động.
- **EC7 – Password rỗng:** Người dùng không nhập password.
- **EC8 – Password không hợp lệ:** Password không khớp với thông tin đăng nhập.

Table 3: Các test case EP cho FR-01

Mã TC	**Phân lớp**	**Giá trị đại diện**	**Kết quả mong đợi**
TC-LOGIN-EP1	EC1	standard_user/secret_sauce	Đăng nhập thành công; chuyển đến `inventory.html`.
TC-LOGIN-EP2	EC2	locked_out_user/secret_sauce	Hiển thị `Sorry, this user has been locked out.`
TC-LOGIN-EP3	EC3	””/secret_sauce	Hiển thị `Username is required.`
TC-LOGIN-EP4	EC4	abc_user/secret_sauce	Hiển thị thông báo `Username and password do not match any user in this service.`
TC-LOGIN-EP5	EC5	Standard_User/secret_sauce	Hiển thị thông báo `Username and password do not match any user in this service.`
TC-LOGIN-EP6	EC6	problem_user/secret_sauce	Đăng nhập thành công; chuyển đến `inventory.html`.
TC-LOGIN-EP7	EC7	standard_user/””	Hiển thị `Password is required.`
TC-LOGIN-EP8	EC8	standard_user/wrong_password	Hiển thị thông báo `Username and password do not match.`

3.1.2 EP – Phân lớp tương đương cho FR-02

Đối với FR-02, trạng thái của một sản phẩm được chia thành hai lớp tương đương dựa trên việc sản phẩm đã có trong giỏ hàng hay chưa. Mỗi lớp đại diện cho một trạng thái của nút thao tác và có hành vi tương ứng khi người dùng nhấn nút. Các test case được thiết kế để kiểm tra hành vi của từng lớp, đồng thời xác minh sự thay đổi trạng thái sau thao tác.

- **EC1 – Sản phẩm chưa có trong giỏ hàng:** Nút hiển thị `Add to cart`. Khi nhấn nút, sản phẩm được thêm vào giỏ hàng.
- **EC2 – Sản phẩm đã có trong giỏ hàng:** Nút hiển thị `Remove`. Khi nhấn nút, sản phẩm bị xóa khỏi giỏ hàng.

Table 4: Các test case EP cho FR-02

Mã TC	Phân lớp	giá trị đại diện	Kết quả mong đợi
TC-CART-EP1	EC1	nút sản phẩm hiển thị `Add to cart`	Sản phẩm được thêm vào giỏ khi nhấn; nút chuyển thành `Remove`; badge tăng lên 1.
TC-CART-EP2	EC2	nút sản phẩm hiển thị `Remove`	Sản phẩm bị xóa khỏi giỏ khi nhấn; nút chuyển thành `Add to cart`; badge giảm đi 1 hoặc biến mất nếu giỏ hàng trở nên rỗng.

3.1.3 EP – Phân lớp tương đương cho FR-04

Đối với FR-04, đầu vào là kiểu sắp xếp được chọn trong dropdown ở trang `inventory.html`. Mỗi lựa chọn được hệ thống xử lý theo một quy tắc riêng nên là một lớp tương đương.

- **EC1 – Name (A to Z):** tên sản phẩm xếp tăng dần theo bảng chữ cái.
- **EC2 – Name (Z to A):** tên sản phẩm xếp giảm dần theo bảng chữ cái.
- **EC3 – Price (low to high):** giá xếp tăng dần theo giá trị số.
- **EC4 – Price (high to low):** giá xếp giảm dần theo giá trị số.

Table 5: Các test case EP cho FR-04

Mã TC	Phân lớp	Cách thực hiện	Kết quả mong đợi
TC-SORT-01	EC1	Đăng nhập `standard_user/secret_sauce`, chọn `Name (A to Z)`	Đủ 6 sản phẩm; tên xếp tăng dần theo bảng chữ cái.
TC-SORT-02	EC2	Đăng nhập `standard_user/secret_sauce`, chọn `Name (Z to A)`	Đủ 6 sản phẩm; tên xếp giảm dần theo bảng chữ cái.
TC-SORT-03	EC3	Đăng nhập `standard_user/secret_sauce`, chọn `Price (low to high)`	Đủ 6 sản phẩm; giá xếp tăng dần theo giá trị số.
TC-SORT-04	EC4	Đăng nhập `standard_user/secret_sauce`, chọn `Price (high to low)`	Đủ 6 sản phẩm; giá xếp giảm dần theo giá trị số.
TC-SORT-05	EC2, EC3, EC4	Đăng nhập `problem_user/secret_sauce`, lần lượt chọn `Name (Z to A)`, `Price (low to high)`, `Price (high to low)`	Mỗi lần chọn, danh sách sắp xếp theo đúng kiểu đã chọn (như TC-SORT-02..04).
TC-SORT-06	EC2, EC3, EC4	Đăng nhập `visual_user/secret_sauce`, lần lượt chọn `Name (Z to A)`, `Price (low to high)`, `Price (high to low)`	Danh sách xếp đúng kiểu đã chọn; tên và giá của từng sản phẩm không thay đổi so với thứ tự mặc định.

3.1.4 Chi tiết các test case EP

TC-CART-EP1 – Thêm sản phẩm chưa có trong giỏ

Kỹ thuật: EP

Mục tiêu: Kiểm tra hành vi thêm sản phẩm thuộc lớp tương đương EC1.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; giỏ hàng đang rỗng.

Dữ liệu kiểm thử: Sauce Labs Backpack.

Các bước thực hiện:
1. Mở trang danh sách sản phẩm.
2. Xác nhận nút của Sauce Labs Backpack hiển thị `Add to cart`.
3. Nhấn `Add to cart`.
4. Quan sát nút sản phẩm và badge giỏ hàng.

Kết quả mong đợi: Sản phẩm được thêm vào giỏ hàng; nút chuyển thành `Remove`; badge hiển thị số 1.

TC-CART-EP2 – Xóa sản phẩm đã có trong giỏ

Kỹ thuật: EP

Mục tiêu: Kiểm tra hành vi xóa sản phẩm thuộc lớp tương đương EC2.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; Sauce Labs Backpack đã có trong giỏ hàng.

Dữ liệu kiểm thử: Sauce Labs Backpack.

Các bước thực hiện:
1. Mở trang danh sách sản phẩm.
2. Xác nhận nút của Sauce Labs Backpack hiển thị `Remove`.
3. Nhấn `Remove`.
4. Quan sát nút sản phẩm và badge giỏ hàng.

Kết quả mong đợi: Sản phẩm bị xóa khỏi giỏ hàng; nút chuyển thành `Add to cart`; số lượng trong giỏ hàng giảm đi 1 hoặc trở nên rỗng dẫn tới badge không hiển thị.

TC-SORT-01 – Sắp xếp theo tên A đến Z

Kỹ thuật: EP

Mục tiêu: Kiểm tra hành vi sắp xếp thuộc lớp tương đương EC1.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; đang ở trang `inventory.html`.

Dữ liệu kiểm thử: Option = Name (A to Z) (az).

Các bước thực hiện:
1. Chọn `Name (Z to A)` trong dropdown để đổi khỏi kiểu mặc định.
2. Chọn `Name (A to Z)` trong dropdown.
3. Đọc danh sách tên sản phẩm.

Kết quả mong đợi: Dropdown hiển thị `Name (A to Z)`; có đủ 6 sản phẩm; tên xếp tăng dần theo bảng chữ cái.

TC-SORT-02 – Sắp xếp theo tên Z đến A

Kỹ thuật: EP

Mục tiêu: Kiểm tra hành vi sắp xếp thuộc lớp tương đương EC2.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; đang ở trang `inventory.html`.

Dữ liệu kiểm thử: Option = Name (Z to A) (za).

Các bước thực hiện:
1. Chọn `Name (Z to A)` trong dropdown.
2. Đọc danh sách tên sản phẩm.

Kết quả mong đợi: Dropdown hiển thị `Name (Z to A)`; đủ 6 sản phẩm; tên xếp giảm dần theo bảng chữ cái.

TC-SORT-03 – Sắp xếp theo giá thấp đến cao

Kỹ thuật: EP

Mục tiêu: Kiểm tra hành vi sắp xếp thuộc lớp tương đương EC3.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; đang ở trang `inventory.html`.

Dữ liệu kiểm thử: Option = Price (low to high) (lohi).

Các bước thực hiện: 1. Chọn `Price (low to high)` trong dropdown.

2. Đọc danh sách giá và chuyển thành số.

Kết quả mong đợi: Dropdown hiển thị `Price (low to high)`; đủ 6 sản phẩm; giá xếp tăng dần theo giá trị số (không so sánh như chuỗi).

TC-SORT-04 – Sắp xếp theo giá cao đến thấp

Kỹ thuật: EP

Mục tiêu: Kiểm tra hành vi sắp xếp thuộc lớp tương đương EC4.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; đang ở trang `inventory.html`.

Dữ liệu kiểm thử: Option = Price (high to low) (hilo).

Các bước thực hiện: 1. Chọn `Price (high to low)` trong dropdown.

2. Đọc danh sách giá và chuyển thành số.

Kết quả mong đợi: Dropdown hiển thị `Price (high to low)`; đủ 6 sản phẩm; giá xếp giảm dần theo giá trị số.

TC-SORT-05 – Sắp xếp với tài khoản problem_user

Kỹ thuật: EP

Mục tiêu: Kiểm tra các lớp EC2, EC3, EC4 với tài khoản có lỗi giao diện cài sẵn.

Tiền điều kiện: Người dùng đã đăng nhập bằng `problem_user`; đang ở `inventory.html`.

Dữ liệu kiểm thử: Lần lượt za, lohi, hilo.

Các bước thực hiện: 1. Chọn `Name (Z to A)`, đọc danh sách tên.

2. Chọn `Price (low to high)`, đọc danh sách giá.

3. Chọn `Price (high to low)`, đọc danh sách giá.

Kết quả mong đợi: Sau mỗi bước, danh sách đổi thứ tự đúng kiểu đã chọn; đủ 6 sản phẩm.

TC-SORT-06 – Sắp xếp với visual_user

Kỹ thuật: EP

Mục tiêu: Kiểm tra tên và giá không bị thay đổi, và sắp xếp đúng khi dùng tài khoản bổ sung `visual_user`.

Tiền điều kiện: Người dùng đã đăng nhập bằng `visual_user`; đang ở `inventory.html`; đã ghi lại tên và giá ở thứ tự mặc định.

Dữ liệu kiểm thử: Lần lượt za, lohi, hilo.

Các bước thực hiện:
1. Chọn `Name (Z to A)`, đọc tên và giá.
2. Chọn `Price (low to high)`, đọc danh sách giá.
3. Chọn `Price (high to low)`, đọc danh sách giá.

Kết quả mong đợi: Tên và giá mỗi sản phẩm giữ nguyên so với mặc định; sau mỗi bước danh sách xếp đúng kiểu đã chọn.

3.2 Phân tích giá trị biên (BVA)

3.2.1 BVA – Phân tích giá trị biên cho FR-01

Miền giá trị được xét từ -1 đến 2. -1 khi tài khoản không tồn tại hoặc sai mật khẩu, 0 khi giá trị rỗng, +1 khi đúng tên đăng nhập hoặc mật khẩu. khi đạt giá trị 2 thì đăng nhập thành công và chuyển đến

Table 6: Các test case BVA cho FR-01

Mã TC	Giá trị	Cách thực hiện	Kết quả mong đợi
TC-LOGIN-B1	-1	Username = abc_user, password = secret_sauce	Hiển thị thông báo thông tin đăng nhập không hợp lệ.
TC-LOGIN-B2	-1	Username = standard_user, password = wrong_password	Hiển thị thông báo thông tin đăng nhập không hợp lệ.
TC-LOGIN-B3	0 (min)	Để trống cả username và password	Hiển thị Username is required.
TC-LOGIN-B4	1 (min+1)	Username = standard_user, để trống password	Hiển thị Password is required.
TC-LOGIN-B5	1 (max-1)	Để trống username, password = secret_sauce	Hiển thị Username is required.
TC-LOGIN-B6	2 (max)	Username = standard_user, password = secret_sauce	Đăng nhập thành công

3.2.2 BVA – Phân tích giá trị biên cho FR-02

Miền giá trị được xét là từ 0 đến 6 sản phẩm. Khi giỏ hàng rỗng, badge không hiển thị; khi có từ 1 đến 6 sản phẩm, badge hiển thị số lượng tương ứng.

Table 7: Các test case BVA cho FR-02

Mã TC	Giá trị	Cách thực hiện	Kết quả mong đợi
TC-CART-B1	0	Đăng nhập, không thêm sản phẩm	Badge không hiển thị.
TC-CART-B2	1	Thêm một sản phẩm	Badge hiển thị số 1.
TC-CART-B3	5	Thêm 6 sản phẩm, xóa 1 sản phẩm	Badge hiển thị số 5.
TC-CART-B4	6	Thêm cả 6 sản phẩm	Badge hiển thị số 6.
TC-CART-B5	$1 \rightarrow 0$	Thêm 1 sản phẩm, sau đó xóa sản phẩm đó	Badge biến mất.

Chú Ý:Các giá trị biên ngoài miền hợp lệ (-1 và 7) không thể thử được vì giao diện SauceDemo chỉ cung cấp một nút 'Add to cart' cho mỗi sản phẩm và không cho phép thêm cùng một sản phẩm nhiều lần. Vì vậy, nhóm không thể tạo trực tiếp trạng thái giỏ hàng có -1 hoặc 7 sản phẩm thông qua giao diện người dùng bình thường, thế nên chỉ có thể thử từ 0-6.

3.2.3 Chi tiết các test case BVA

TC-CART-B1 – Giỏ hàng rỗng

Kỹ thuật: BVA

Mục tiêu: Kiểm tra trạng thái biên nhỏ nhất của miền giá trị, khi giỏ hàng có 0 sản phẩm.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; giỏ hàng đang rỗng.

Dữ liệu kiểm thử: 0 sản phẩm.

Các bước thực hiện:
1. Mở trang danh sách sản phẩm.
2. Không thêm sản phẩm nào vào giỏ hàng.
3. Quan sát biểu tượng giỏ hàng.

Kết quả mong đợi: Badge số lượng sản phẩm không hiển thị; các nút sản phẩm hiển thị `Add to cart`.

TC-CART-B2 – Giỏ hàng có một sản phẩm

Kỹ thuật: BVA

Mục tiêu: Kiểm tra số lượng tối thiểu lớn hơn 0 của miền giá trị hợp lệ.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; giỏ hàng đang rỗng.

Dữ liệu kiểm thử: Sauce Labs Backpack.

Các bước thực hiện:
1. Mở trang danh sách sản phẩm.
2. Nhấn `Add to cart` tại Sauce Labs Backpack.
3. Quan sát nút của sản phẩm và badge giỏ hàng.

Kết quả mong đợi: Nút chuyển thành `Remove`; badge hiển thị số 1.

TC-CART-B3 – Giỏ hàng có năm sản phẩm

Kỹ thuật: BVA

Mục tiêu: Kiểm tra giá trị ngay dưới giới hạn tối đa của miền giá trị hợp lệ.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; giỏ hàng đang rỗng.

Dữ liệu kiểm thử: Sáu sản phẩm có sẵn trên trang danh sách sản phẩm.

Các bước thực hiện:
1. Thêm cả sáu sản phẩm vào giỏ hàng.
2. Nhấn `Remove` tại một sản phẩm bất kỳ.
3. Quan sát badge giỏ hàng.

Kết quả mong đợi: Sản phẩm bị xóa có nút `Add to cart`; badge hiển thị số 5.

TC-CART-B4 – Giỏ hàng có sáu sản phẩm

Kỹ thuật: BVA

Mục tiêu: Kiểm tra giới hạn tối đa của miền giá trị hợp lệ.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; giỏ hàng đang rỗng.

Dữ liệu kiểm thử: Cả sáu sản phẩm có sẵn trên trang danh sách sản phẩm.

Các bước thực hiện:
1. Thêm lần lượt cả sáu sản phẩm vào giỏ hàng.
2. Quan sát badge giỏ hàng.

Kết quả mong đợi: Badge hiển thị số 6; cả sáu nút sản phẩm chuyển thành `Remove`.

TC-CART-B5 – Xóa sản phẩm cuối cùng

Kỹ thuật: BVA (kiểm tra chuyển trạng thái)

Mục tiêu: Xác minh badge biến mất khi số lượng sản phẩm giảm từ 1 xuống 0.

Tiền điều kiện: Người dùng đã đăng nhập bằng `standard_user`; giỏ hàng đang rỗng.

Dữ liệu kiểm thử: Sauce Labs Backpack.

Các bước thực hiện:
1. Thêm Sauce Labs Backpack vào giỏ hàng.
2. Xác nhận badge hiển thị số 1.
3. Nhấn `Remove` tại sản phẩm đó.
4. Quan sát badge và nút sản phẩm.

Kết quả mong đợi: Badge không còn hiển thị; nút sản phẩm trở về `Add to cart`.

3.3 bảng quyết định(DT)

3.3.1 DT – Bảng quyết định cho FR-01

Bảng quyết định kiểm tra các tổ hợp điều kiện đăng nhập, bao gồm username rỗng, password rỗng, thông tin đăng nhập đúng và trạng thái tài khoản bị khóa. Ký hiệu Y là đúng, N là sai và – là không ảnh hưởng.

Table 8: Bảng quyết định cho FR-01

Điều kiện / Hành động	**R1**	**R2**	**R3**	**R4**	**R5**
C1: Username rỗng	Y	N	N	N	N
C2: Password rỗng	–	Y	N	N	N
C3: Cặp username/password đúng	–	–	N	Y	Y
C4: Tài khoản bị khóa	–	–	–	Y	N
A1: Username is required	X				
A2: Password is required		X			
A3: Username and password do not match...			X		
A4: Sorry, this user has been locked out.				X	
A5: Chuyển đến inventory.html					X

Table 9: Các quy tắc và dữ liệu kiểm thử cho FR-01

Quy tắc	**Mã TC**	**Username**	**Password**
R1	TC-LOGIN-DT1	""	""
R2	TC-LOGIN-DT2	standard_user	""
R3	TC-LOGIN-DT3	locked_out_user	wrong_password
R4	TC-LOGIN-DT4	locked_out_user	secret_sauce
R5	TC-LOGIN-DT5	standard_user	secret_sauce

3.3.2 DT – Bảng quyết định cho FR-03

Bản quyết định kiểm tra các tổ hợp điều kiện để checkout, bao gồm `First Name` rỗng, `Last Name` rỗng, `Postal Code` rỗng. Ký hiệu Y là đúng, N là sai và - là không ảnh hưởng

Table 10: Bảng quyết định cho FR-03

Điều kiện / Hành động	**R1**	**R2**	**R3**	**R4**
C1:First Name rỗng	Y	N	N	N
C2: Last Name rỗng	-	Y	N	N
C3: Postal Code rỗng	-	-	Y	N
C4: nhập hết các trường	-	-	-	N
A1: First Name is Required	X	-	-	-
A2: Last Name is Required	-	X	-	-
A3: Postal Code is Required	-	-	X	-
A4: Thank you for your order!	-	-	-	X

Table 11: Các quy tắc và dữ liệu thử cho FR-03

Quy tắc	**Mã TC**	**First Name**	**Last Name**	**Postral Code**
R1	TC-CHK-DC02	""	Nguyen	700000
R2	TC-CHK-DC03	An	""	700000
R3	TC-CHK-DC04	An	Nguyen	""
R4	TC-CHK-DC01	An	Nguyen	700000

3.4 kiểm thử ca sử dụng(UC)

3.4.1 UC – Kiểm thử ca sử dụng cho FR-01

Tên ca sử dụng: Đăng nhập (Login)

Tác nhân: Người dùng.

Tiền điều kiện: Người dùng truy cập trang đăng nhập SauceDemo.

Hậu điều kiện:

- Nếu đăng nhập thành công, hệ thống chuyển đến `inventory.html`.
- Nếu đăng nhập thất bại, hệ thống hiển thị thông báo lỗi tương ứng và không chuyển đến trang sản phẩm.

Luồng chính:

1. Người dùng mở trang đăng nhập SauceDemo.
2. Người dùng nhập username và password hợp lệ.
3. Người dùng nhấn nút Login.
4. Hệ thống xác thực thông tin đăng nhập.
5. Hệ thống chuyển người dùng đến `inventory.html`.

Các luồng thay thế:

- **A1 – Username rỗng:** Người dùng để trống username và nhấn Login. Hệ thống hiển thị `Username is required`.

- **A2 – Password rỗng:** Người dùng nhập username nhưng để trống password và nhấn Login. Hệ thống hiển thị `Password is required`.
- **A3 – Thông tin đăng nhập không hợp lệ:** Người dùng nhập username không tồn tại, username sai chữ hoa/thường hoặc password không chính xác. Hệ thống hiển thị `Username and password do not match any user in this service`.
- **A4 – Tài khoản bị khóa:** Người dùng nhập đúng thông tin của tài khoản bị khóa. Hệ thống hiển thị `Sorry, this user has been locked out.`

Table 12: Các test case UC cho FR-01

Mã TC	**Luồng**	**Username / Password**	**Kết quả mong đợi**
TC-LOGIN-EP1	Luồng chính	standard_user / secret_sauce	Đăng nhập thành công; chuyển đến `inventory.html`.
TC-LOGIN-EP3	A1	"" / secret_sauce	Hiển thị `Username is required`.
TC-LOGIN-EP7	A2	standard_user / ""	Hiển thị `Password is required`.
TC-LOGIN-EP4	A3	abc_user / secret_sauce	Hiển thị thông báo thông tin đăng nhập không hợp lệ.
TC-LOGIN-EP5	A3	Standard_User / secret_sauce	Hiển thị thông báo thông tin đăng nhập không hợp lệ.
TC-LOGIN-EP8	A3	standard_user / wrong_password	Hiển thị thông báo thông tin đăng nhập không hợp lệ.
TC-LOGIN-EP2	A4	locked_out_user / secret_sauce	Hiển thị `Sorry, this user has been locked out.`

3.5 UC–Kiểm thử ca sử dụng cho FR-03

- **Mã/Tên** : TC-CHK-01.
- **Yêu cầu/ Kỹ Thuật** : FR-03 / use case, luồng chính.
- **Tiền điều kiện** Đã đăng nhập bằng standard_user ,giỏ hàng rỗng.
- **Dữ Liệu** : First Name = An, Last Name = Nguyen, Postal Code = 70000.

Table 13: Đặt tả chi tiết một test case

Bước	**Thao Tác**	**Kết quả mong đợi**
1	Nhấn Add to cart ở Sauce Labs Backpack	Badge giỏ hàng = 1
2	Nhấn biểu tượng giỏ hàng	URL cart.html; có 1 sản phẩm giá $29
3	Nhấn Checkout	URL checkout-step-one.html
4	Nhập dữ liệu, nhấn Continue	URL checkout-step-two.html
5	Đọc phần tổng tiền	Item total: $29.99, Tax: $2.40, Total: $
6	Nhấn Finish	URL checkout-complete.html; Thank you for your orde

Table 14: Các test case UC cho FR-03

Mã TC	Luồng	First / Last / Postal	Kết quả mong đợi
TC-CHK-01	Luồng Chính	An / Nguyen / 700000	Thank you for your order!
TC-CHK-02	A1	"" / Nguyen / 700000	Error: First Name is required
TC-CHK-03	A2	An / "" / 700000	Error: Last Name is required
TC-CHK-04	A3	An / Nguyen / ""	Error: Postal Code is required
TC-CHK-05	A4	(không nhập) / Cancel	URL cart.html, sản phẩm vẫn còn trong giỏ

3.6 NFR-01 Hiệu năng

Table 15: Test case TC-NFR-01: Đo thời gian hiển thị trang sản phẩm khi đăng nhập

Mã TC: TC-NFR-01
Mục tiêu: Trang sản phẩm hiển thị trong $\leq$ 3s khi đăng nhập thành công
Kỹ thuật: NFR-01 / Đo thời gian
Tiền điều kiện: Chưa đăng nhập, đã xóa cookie và cache, kết nối mạng ổn định
Dữ liệu kiểm thử: `performance_glitch_user`, thực hiện đo 5 lần

Bước	Thao tác	Kết quả mong đợi
1	Truy cập https://www.saucedemo.com	Hiển thị form đăng nhập
2	Nhập thông tin `Username` và `Password`	Dữ liệu được nhập chính xác vào các ô tương ứng
3	Mở DevTools (`F12`) $\rightarrow$ tab **Performance** (bật Screenshots), nhấn **Record**, nhấn nút **Login**, đợi danh sách sản phẩm hiển thị hoàn tất thì bấm **Stop**; đo thời gian từ sự kiện Click đến khung hình xuất hiện danh sách; ghi nhận kết quả lần 1 và chụp ảnh minh chứng	Chuyển hướng thành công đến `inventory.html`; ghi nhận được thời gian phản hồi lần 1
4	Xóa cookie, lặp lại các bước 1–3 thêm 4 lần	Thu thập đủ bộ dữ liệu gồm 5 giá trị đo
5	Sắp xếp 5 giá trị đo theo thứ tự tăng dần, lấy giá trị ở vị trí thứ 3 (trung vị)	Giá trị trung vị (median) $\leq$ 3.0s

3.7 Tổng số test case

Bảng dưới đây tổng hợp số lượng test case được thiết kế theo từng kỹ thuật kiểm thử. Các thành viên bổ sung mã test case và số lượng thực tế sau khi hoàn thành từng phần. Mỗi test case được xác định bằng một mã duy nhất để thuận tiện cho việc truy vết và tránh đếm trùng.

Table 16: Tổng hợp số lượng test case theo kỹ thuật

STT	Kỹ thuật	Phạm vi áp dụng	Tối thiểu	Thực tế
1	EP – Phân lớp tương đương	2 trường dữ liệu	8	...
2	BVA – Phân tích giá trị biên	2 miền giá trị	8	...
3	DT – Bảng quyết định	2 bảng quyết định	8	...
4	UC – Kiểm thử theo ca sử dụng	2 ca sử dụng, mỗi ca có ít nhất 2 luồng thay thế	8	...
5	NFR-01 – Hiệu năng	Kiểm tra thời gian tải trang sau đăng nhập	1	...
		Tổng số test case	**39**	**...**

Lưu ý: Cột "Thực tế" ghi số test case đã thiết kế cho từng kỹ thuật. Nếu một test case được sử dụng cho nhiều kỹ thuật, cần đối chiếu mã test case để tránh đếm trùng khi tính tổng số test case duy nhất.

4 NV4 – Thực thi thủ công và báo lỗi

4.1 Ghi kết quả thực thi

Mỗi lần thực thi test case được ghi nhận bằng một dòng. Trạng thái thực thi gồm PASS, FAIL hoặc BLOCKED. Kết quả phải phản ánh quá trình kiểm thử thực tế của nhóm.

Table 17: Kết quả thực thi test case

Mã TC	**Ngày / Người thực hiện**	**Môi trường**	**Kết quả thực tế**	**Trạng thái**
TC-LOGIN-EP1	19/10 - Quý	Edge 155 win11	chuyển đến trang inventory.html	PASS
TC-LOGIN-EP2	19/10 - Quý	Edge 155 win11	hiện thông báo sorry, this user has been locked	PASS
TC-LOGIN-EP3	19/10 - Quý	Edge 155 win11	hiện thông báo username is required	PASS
TC-LOGIN-EP4	19/10 - Quý	Edge 155 win11	hiện thông báo Username and password do not match any user in this service	PASS
TC-LOGIN-EP5	19/10 - Quý	Edge 155 win11C	hiện thông báo Username and password do not match any user in this service	PASS
TC-LOGIN-EP7	19/10 - Quý	Edge 155 win11	hiện thông báo password is required	PASS
TC-LOGIN-EP8	19/10 - Quý	Edge 155 win11	hiện thông báo Username and password do not match any user in this service	PASS
TC-LOGIN-DT1	19/10 - Quý	Edge 155 win11	hiện thông báo username is required	PASS

Table 18: Kết quả thực thi test case

Mã TC	**Ngày / Người thực hiện**	**Môi trường**	**Kết quả thực tế**	**Trạng thái**
TC-CART-B1	15/10 - Nhân	Chrome 155 win10 LTSC	số lượng sản phẩn badge không hiển thị, nút add to cart hiển thị đầy đủ trên 6 sản phẩm	PASS
TC-CART-B2	15/10 - Nhân	Chrome 155 win10 LTSC	badge hiện số 1	PASS
TC-CART-B4	15/10 - Nhân	Chrome 155 win10 LTSC	badge hiện số 6	PASS
TC-CART-B3	15/10 - Nhân	Chrome 155 win10 LTSC	badge hiện số 5	PASS
TC-CART-B5	15/10 - Nhân	Chrome 155 win10 LTSC	badge biến mất	PASS
TC-CART-EP1	15/10 - Nhân	Chrome 155 win10 LTSC	badge hiện số 1, nút sản phẩm đổi thành Remove	PASS
TC-CART-EP2	15/10 - Nhân	Chrome 155 win10 LTSC	nút sản phẩm chuyển sang Remove và giỏ hàng badge giảm đi 1	PASS

Table 19: Kết quả thực thi test case FR-03

Mã TC	**Ngày / Người thực hiện**	**Môi trường**	**Kết quả thực tế**	**Trạng thái**
TC-CHK-UC01	19/10 - Thiện	Chrome 155 win11	Total = $103.65, Hiện Thank	PASS
TC-CHK-UC02	19/10 - Thiện	Chrome 155 win11	hiện thông báo First Name is required	PASS
TC-CHK-UC03	19/10 - Thiện	Chrome 155 win11	hiện thông báo Last Name is required	PASS
TC-CHK-UC04	19/10 - Thiện	Chrome 155 win11	hiện thông báo Postal Code is required	PASS
TC-CHK-UC05	19/10 - Thiện	Chrome 155 win11	Quay về cart.html, sản phẩm còn trong giỏ	PASS

Table 20: Kết quả thực thi test case FR-04

Mã TC	Ngày / Người thực hiện	Môi trường	Kết quả thực tế	Trạng thái
TC-SORT-01	20/10 – Duy	Chrome 155, Windows 11 (`standard_user`)	Dropdown đổi đúng kiểu; tên/giá xếp đúng quy tắc, đủ 6 sản phẩm	PASS
TC-SORT-02	20/10 – Duy	Chrome 155, Windows 11 (`standard_user`)	Dropdown đổi đúng kiểu; tên/giá xếp đúng quy tắc, đủ 6 sản phẩm	PASS
TC-SORT-03	20/10 – Duy	Chrome 155, Windows 11 (`standard_user`)	Dropdown đổi đúng kiểu; tên/giá xếp đúng quy tắc, đủ 6 sản phẩm	PASS
TC-SORT-04	20/10 – Duy	Chrome 155, Windows 11 (`standard_user`)	Dropdown đổi đúng kiểu; tên/giá xếp đúng quy tắc, đủ 6 sản phẩm	PASS
TC-SORT-05	20/10 – Duy	Chrome 155, Windows 11 (`problem_user`)	Mặc định A-Z đúng; chọn Z-A, lohi, hilo thì danh sách giữ nguyên	FAIL (BUG-02)
TC-SORT-06	20/10 – Duy	Chrome 155, Windows 11 (`visual_user`)	Chọn Z-A: tên đúng thứ tự nhưng giá bị đổi; chọn lohi/hilo: giá không xếp theo thứ tự	FAIL (BUG-03)

4.1.1 NFR-01

Table 21: Kết quả đo thời gian tải trang sản phẩm (`inventory.html`)

Môi trường kiểm thử: Windows 11; Chrome 155.0.8059.40; Java 21.0.10; mạng [ĐIỀN]; ngày [ĐIỀN]
Cách đo: Dùng tab Performance của Chrome DevTools, bấm **Login**, đợi trang `inventory.html` hiển thị hoàn tất rồi ghi nhận thời gian.

Lần đo	1	2	3	4	5	Trung vị	Trạng thái
Thời gian	0.20s	0.10s	0.09s	0.10s	0.11s	0.10s	**PASS**

4.2 Báo cáo lỗi

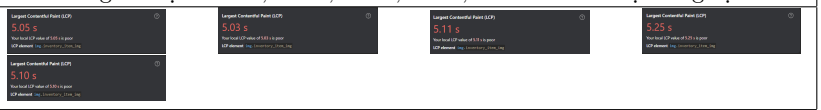
Mỗi lỗi được phát hiện cần có mã định danh duy nhất. Các test case liên quan đến cùng một lỗi sẽ tham chiếu đến mã lỗi đó.

Table 22: Báo cáo lỗi

Trường thông tin	**Nội dung**
Mã lỗi / Tiêu đề	
Mức nghiêm trọng	
Môi trường	
Tiền điều kiện	
Bước tái hiện	
Kết quả mong đợi	
Kết quả thực tế	
Bằng chứng	

4.2.1 NFR-01

Table 23: Báo cáo lỗi BUG-01: Hiệu năng tải trang sản phẩm vượt quá giới hạn cho phép

Mã / Tiêu đề	**BUG-01** – Trang sản phẩm mất khoảng 5 giây để hiển thị với `performance_glitch_user`
Mức nghiêm trọng	**Major** – Vi phạm NFR-01, người dùng có thể tưởng hệ thống bị treo
Môi trường	Windows 11, Chrome 155.0.8059.40, mạng ổn định
Tiền điều kiện	Chưa đăng nhập, đã xóa cookie và cache trình duyệt
Bước tái hiện	1. Truy cập https://www.saucedemo.com 2. Nhập thông tin tài khoản: `performance_glitch_user` / `secret_sauce` 3. Mở tab Performance (DevTools), nhấn Login, đợi trang `inventory.html` hiển thị hoàn tất rồi ghi nhận thời gian
Kết quả mong đợi	Trang sản phẩm hiển thị hoàn tất trong vòng $\leq$ 3s
Kết quả thực tế	5 lần đo ghi nhận: 5.05s, 5.03s, 5.11s, 5.25s, 5.10s $\rightarrow$ Giá trị trung vị: 5.10s
Bằng chứng	

4.2.2 FR-04

Table 24: Báo cáo lỗi BUG-02: Dropdown sắp xếp không có tác dụng với problem_user

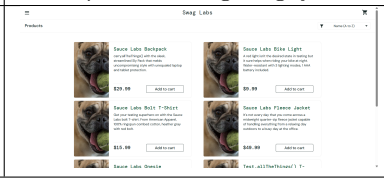
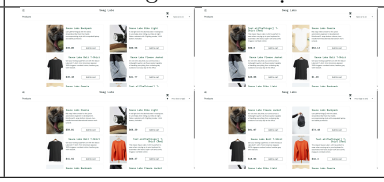
Mã / Tiêu đề	**BUG-02** – Dropdown sắp xếp không có tác dụng với `problem_user` (chọn Z-A, Price low-high, Price high-low)
Mức nghiêm trọng	**Major** – 3 trong 4 kiểu sắp xếp của FR-04 không hoạt động
Môi trường	Chrome 155, Windows 11, ngày 20/10
Tiền điều kiện	Chưa đăng nhập, xóa cookie
Bước tái hiện	1. Truy cập https://www.saucedemo.com 2. Đăng nhập `problem_user` / `secret_sauce` 3. Ở trang `inventory.html`, chọn Name (Z to A) (lặp lại với Price low to high, Price high to low)
Kết quả mong đợi	Danh sách sắp xếp lại theo kiểu đã chọn
Kết quả thực tế	Thứ tự danh sách giữ nguyên như mặc định A-Z
Bằng chứng	

Table 25: Báo cáo lỗi BUG-03: Giá sản phẩm bị thay đổi và không xếp đúng thứ tự khi sắp xếp với visual_user

Mã / Tiêu đề	**BUG-03** – Giá sản phẩm bị thay đổi và không xếp đúng thứ tự khi sắp xếp với `visual_user`
Mức nghiêm trọng	**Major** – Hiển thị sai giá có thể gây hiểu nhầm khi chọn mua
Môi trường	Chrome 155, Windows 11, ngày 20/10
Tiền điều kiện	Chưa đăng nhập, xóa cookie
Bước tái hiện	1. Mở https://www.saucedemo.com 2. Đăng nhập `visual_user` / `secret_sauce` 3. Ghi lại giá 6 sản phẩm ở thứ tự mặc định 4. Chọn Name (Z to A), rồi Price (low to high), rồi Price (high to low), mỗi lần ghi lại giá
Kết quả mong đợi	Giá mỗi sản phẩm không đổi; danh sách sắp xếp lại theo kiểu đã chọn
Kết quả thực tế	Chọn Z-A: sắp xếp đúng nhưng giá thay đổi. Chọn sắp theo giá: giá thay đổi và không theo thứ tự.
Bằng chứng	

5 NV5 – Tự động hóa bằng JUnit 5

5.1 phiên bản dự án

Junit Version: 5.11.4
Selenium Version: 4.51.0
SureFire Version: 3.5.2

5.2 Các Test case Trong mã nguồn

Table 26: Các test case được triển khai trong mã nguồn

File	Trách nhiệm	Test case liên quan
`CartTest.java`	Kiểm thử chức năng giỏ hàng, bao gồm số lượng sản phẩm hiển thị trên badge, cập nhật số lượng khi thêm hoặc xóa sản phẩm, và trạng thái nút thêm/xóa sản phẩm. Sử dụng JUnit 5 và `@CsvFileSource` để kiểm thử với dữ liệu đầu vào.	`TC-CART-B1`–`TC-CART-B5`; `TC-CART-EP1`–`TC-CART-EP2`;
`CheckoutTest.java`	Kiểm tra thông báo khi thiếu 1 bộ dữ liệu vả xác nhận vẫn ở trang nhập thông tin, kiểm thông tin hợp lệ và kiểm tra giá trị `Total`, `Tax`, `ItemTotal`, nhấp `Finish` và kiểm tra lời cảm ơn, badge biến mất, kiểm thử nút `Cancel` quay về giỏ hàng và sản phẩm vẫn còn.Sử dụng JUnit 5, `@ParameterizedTest` và `@CsvSource` để kiểm thử với các bộ dữ liệu đầu vào khác nhau.	`TC-CHK-01-05`
`SortTest.java`	Kiểm thử chức năng sắp xếp sản phẩm trên trang `inventory.html`, bao gồm sắp xếp theo tên (A to Z, Z to A) và theo giá (low to high, high to low). Kiểm tra tính nguyên vẹn (đủ 6 sản phẩm, tên và giá không thay đổi) và xử lý với các tài khoản có lỗi đặc biệt (`problem_user`, `visual_user`). Sử dụng JUnit 5, `@ParameterizedTest`, `@CsvSource` và `@MethodSource`.	`TC-SORT-01`–`06`

5.3 Các file

5.3.1 FR-03

Listing 1: CheckoutPage.java

```
package vn.edu.softwaretesting.pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.TimeoutException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public final class CheckoutPage {
    private static final By FIRST = By.id("first-name");
    private static final By LAST = By.id("last-name");
    private static final By POSTAL = By.id("postal-code");
    private static final By CONTINUE = By.id("continue");
    private static final By CANCEL = By.id("cancel");
    private static final By ERROR = By.cssSelector("[data-test='error']");
    private static final By SUBTOTAL = By.cssSelector(".summary_subtotal_label");
    private static final By TAX = By.cssSelector(".summary_tax_label");
    private static final By TOTAL = By.cssSelector(".summary_total_label");
    private static final By FINISH = By.id("finish");
    private static final By HEADER = By.cssSelector(".complete-header");
    private static final By BADGE = By.cssSelector(".shopping_cart_badge");

    private final WebDriver driver;
    private final WebDriverWait wait;

    public CheckoutPage(WebDriver driver) {
        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(8));
    }

    public String getCurrentUrl() {
        return driver.getCurrentUrl();
    }

    public CheckoutPage waitForInformationPage() {
        wait.until(ExpectedConditions.urlContains("checkout-step-one.html"));
        wait.until(ExpectedConditions.visibilityOfElementLocated(FIRST));
        wait.until(ExpectedConditions.visibilityOfElementLocated(LAST));
        wait.until(ExpectedConditions.visibilityOfElementLocated(POSTAL));
        return this;
    }

    private void enter(By locator, String value) {
        String expectedValue = value == null ? "" : value;
        WebElement input = wait.until(ExpectedConditions.elementToBeClickable(
             locator));
        input.clear();
        if (!expectedValue.isEmpty()) {
            input.sendKeys(expectedValue);
        }
        wait.withMessage("Input value must update before continuing checkout")
                .until(d -> expectedValue.equals(d.findElement(locator).
                     getDomProperty("value")));
    }

    public CheckoutPage enterFirstName(String value) {
        enter(FIRST, value);
        return this;
    }

    public CheckoutPage enterLastName(String value) {
        enter(LAST, value);
        return this;
    }
```

```
public CheckoutPage enterPostalCode(String value) {
    enter(POSTAL, value);
    return this;
}

public CheckoutPage fillInformation(String first, String last, String postal)
     {
    return enterFirstName(first).enterLastName(last).enterPostalCode(postal);
}

public CheckoutPage clickContinue() {
    wait.until(ExpectedConditions.elementToBeClickable(CONTINUE)).click();
    wait.until(ExpectedConditions.or(
            ExpectedConditions.visibilityOfElementLocated(ERROR),
            ExpectedConditions.urlContains("checkout-step-two.html")));
    return this;
}

public CartPage clickCancel() {
    wait.until(ExpectedConditions.elementToBeClickable(CANCEL)).click();
    return new CartPage(driver).waitForCart();
}

private String text(By locator) {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).
         getText();
}

public String getErrorMessage() {
    return text(ERROR);
}

public String getItemTotal() {
    return text(SUBTOTAL);
}

public String getTax() {
    return text(TAX);
}

public String getTotal() {
    return text(TOTAL);
}

public String getCompleteHeader() {
    return text(HEADER);
}

public CheckoutPage waitForOverview() {
    wait.until(ExpectedConditions.urlContains("checkout-step-two.html"));
    wait.until(ExpectedConditions.visibilityOfElementLocated(SUBTOTAL));
    return this;
}

public CheckoutPage clickFinish() {
    wait.until(ExpectedConditions.elementToBeClickable(FINISH)).click();
    wait.until(ExpectedConditions.urlContains("checkout-complete.html"));
    wait.until(ExpectedConditions.visibilityOfElementLocated(HEADER));
    return this;
}

public boolean isCartBadgeAbsent() {
    try {
        return wait.until(ExpectedConditions.invisibilityOfElementLocated(
             BADGE));
    } catch (TimeoutException exception) {
        return false;
```

```
        }
    }
}
```

Listing 2: CheckoutTest.java

```
package vn.edu.softwaretesting.tests;

import java.util.List;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import vn.edu.softwaretesting.core.WebTestBase;
import vn.edu.softwaretesting.pages.CartPage;
import vn.edu.softwaretesting.pages.CheckoutPage;
import vn.edu.softwaretesting.pages.InventoryPage;
import vn.edu.softwaretesting.pages.LoginPage;

import static org.junit.jupiter.api.Assertions.assertAll;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

@Tag("web")
public class CheckoutTest extends WebTestBase {
    private CheckoutPage checkout;

    @BeforeEach
    void prepareCheckout() {
        LoginPage.LoginResult result = new LoginPage(driver, baseUrl)
                .open().login("standard_user", "secret_sauce");
        assertTrue(result.accepted(), result.message());

        InventoryPage inventory = new InventoryPage(driver).waitUntilLoaded();
        inventory.addProductToCart("Sauce Labs Backpack");
        checkout = new CartPage(driver).open().clickCheckout();
        assertTrue(checkout.getCurrentUrl().contains("checkout-step-one.html"));
    }

    @Test
    @DisplayName("TC-CHK-01")
    void completeCheckout() {
        checkout.fillInformation("An", "Nguyen", "700000").clickContinue().
             waitForOverview();
        assertAll(
                () -> assertTrue(checkout.getCurrentUrl().contains("checkout-step-
                     two.html")),
                () -> assertEquals("Item total: $29.99", checkout.getItemTotal()),
                () -> assertEquals("Tax: $2.40", checkout.getTax()),
                () -> assertEquals("Total: $32.39", checkout.getTotal()));

        checkout.clickFinish();

        assertAll(
                () -> assertTrue(checkout.getCurrentUrl().contains("checkout-
                     complete.html")),
                () -> assertEquals("Thank you for your order!", checkout.
                     getCompleteHeader()),
                () -> assertTrue(checkout.isCartBadgeAbsent(), "Cart badge must
                     disappear"));
    }

    @DisplayName("TC-CHK-02 / TC-CHK-03 / TC-CHK-04")
```

```
    @ParameterizedTest(name = "{0}: missing field -> {4}")
    @CsvSource({
            "TC-CHK-02, '', Nguyen, 700000, Error: First Name is required",
            "TC-CHK-03, An, '', 700000, Error: Last Name is required",
            "TC-CHK-04, An, Nguyen, '', Error: Postal Code is required"
    })
    void rejectMissingField(String caseId, String first, String last,
                            String postal, String expectedError) {
        checkout.fillInformation(first, last, postal).clickContinue();
        assertAll(
                () -> assertEquals(expectedError, checkout.getErrorMessage(),
                    caseId),
                () -> assertTrue(checkout.getCurrentUrl().contains("checkout-step-
                    one.html"), caseId));
    }

    @Test
    @DisplayName("TC-CHK-05")
    void cancelCheckoutKeepsCart() {
        CartPage cart = checkout.clickCancel();
        assertAll(
                () -> assertTrue(cart.getCurrentUrl().contains("cart.html")),
                () -> assertEquals(List.of("Sauce Labs Backpack"), cart.
                    getProductNames()),
                () -> assertEquals(1, cart.getItemCount()),
                () -> assertEquals(1, cart.getCartBadgeCount()));
    }
}
```

Listing 3: CartPage.java

```
package vn.edu.softwaretesting.pages;

import java.time.Duration;
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.NoSuchElementException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public final class CartPage {
    private static final By CART_LINK = By.cssSelector(".shopping_cart_link");
    private static final By CART_LIST = By.cssSelector(".cart_list");
    private static final By ITEMS = By.cssSelector(".cart_item");
    private static final By ITEM_NAME = By.cssSelector(".inventory_item_name");
    private static final By REMOVE = By.cssSelector("button");
    private static final By BADGE = By.cssSelector(".shopping_cart_badge");
    private static final By CHECKOUT = By.id("checkout");
    private static final By CONTINUE_SHOPPING = By.id("continue-shopping");

    private final WebDriver driver;
    private final WebDriverWait wait;

    public CartPage(WebDriver driver) {
        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(8));
    }

    public CartPage open() {
        wait.until(ExpectedConditions.elementToBeClickable(CART_LINK)).click();
        return waitForCart();
    }

    public CartPage waitForCart() {
```

```
        wait.until(ExpectedConditions.urlContains("cart.html"));
        wait.until(ExpectedConditions.visibilityOfElementLocated(CART_LIST));
        return this;
    }

    public String getCurrentUrl() {
        return driver.getCurrentUrl();
    }

    public List<String> getProductNames() {
        return driver.findElements(ITEMS).stream()
                .map(item -> item.findElement(ITEM_NAME).getText())
                .toList();
    }

    public int getItemCount() {
        return driver.findElements(ITEMS).size();
    }

    public int getCartBadgeCount() {
        List<WebElement> badges = driver.findElements(BADGE);
        return badges.isEmpty() ? 0 : Integer.parseInt(badges.get(0).getText());
    }

    public boolean isCartBadgeAbsent() {
        return driver.findElements(BADGE).isEmpty();
    }

    public CartPage removeProduct(String productName) {
        WebElement item = driver.findElements(ITEMS).stream()
                .filter(row -> row.findElement(ITEM_NAME).getText().equals(
                    productName))
                .findFirst()
                .orElseThrow(() -> new NoSuchElementException(
                        "Product is not in the cart: " + productName));
        int expectedCount = getCartBadgeCount() - 1;

        wait.until(ExpectedConditions.elementToBeClickable(item.findElement(REMOVE
            ))).click();
        wait.until(d -> !getProductNames().contains(productName));
        wait.withMessage("Cart badge must update after removing " + productName)
                .until(d -> getCartBadgeCount() == expectedCount);
        return this;
    }

    public CheckoutPage clickCheckout() {
        wait.until(ExpectedConditions.elementToBeClickable(CHECKOUT)).click();
        return new CheckoutPage(driver).waitForInformationPage();
    }

    public InventoryPage continueShopping() {
        wait.until(ExpectedConditions.elementToBeClickable(CONTINUE_SHOPPING)).
            click();
        return new InventoryPage(driver).waitUntilLoaded();
    }
}
```

6 Tài liệu